Dynamic Programming Problem Solving

- Anikate Koul

Problem 1

Flowers:
https://codeforces.com/problemset/problem/474/D

Main Idea

- Let dp[i] represent number of ways in which Marmot can eat **i** flowers.
- Now dp[i] will be sum of two values dp[i-1] (i.e. just add one **red** flower to the bunch of **i-1** flowers) and dp[i-k] (i.e. add **k** white flowers to the bunch of **i-k** flowers).

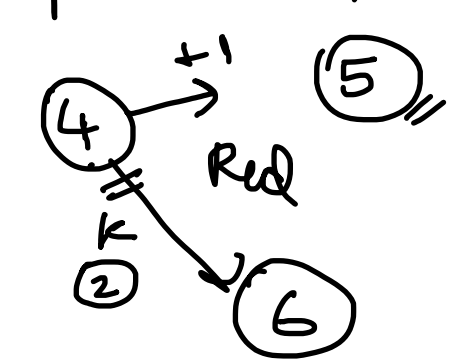
$k = 2$, $n = 4$.

{RRRR}, {WWRR}, {RRWW},
{RWWR}, {WWWW}.

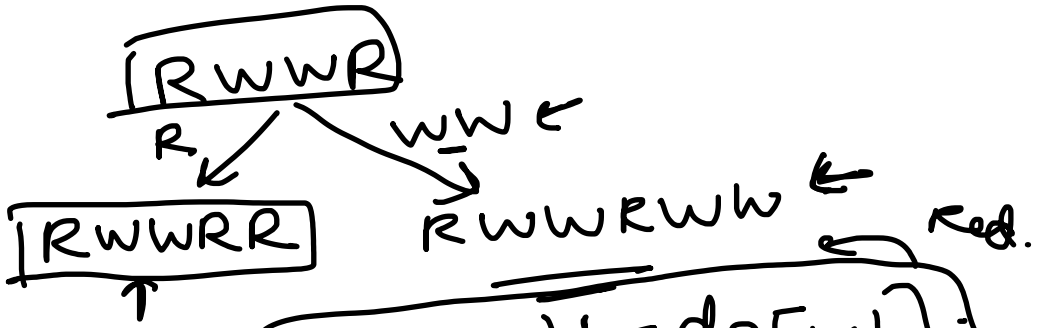
dp[i] → no. of ways to eat i flowers.

{ }

dp[0] → 1.

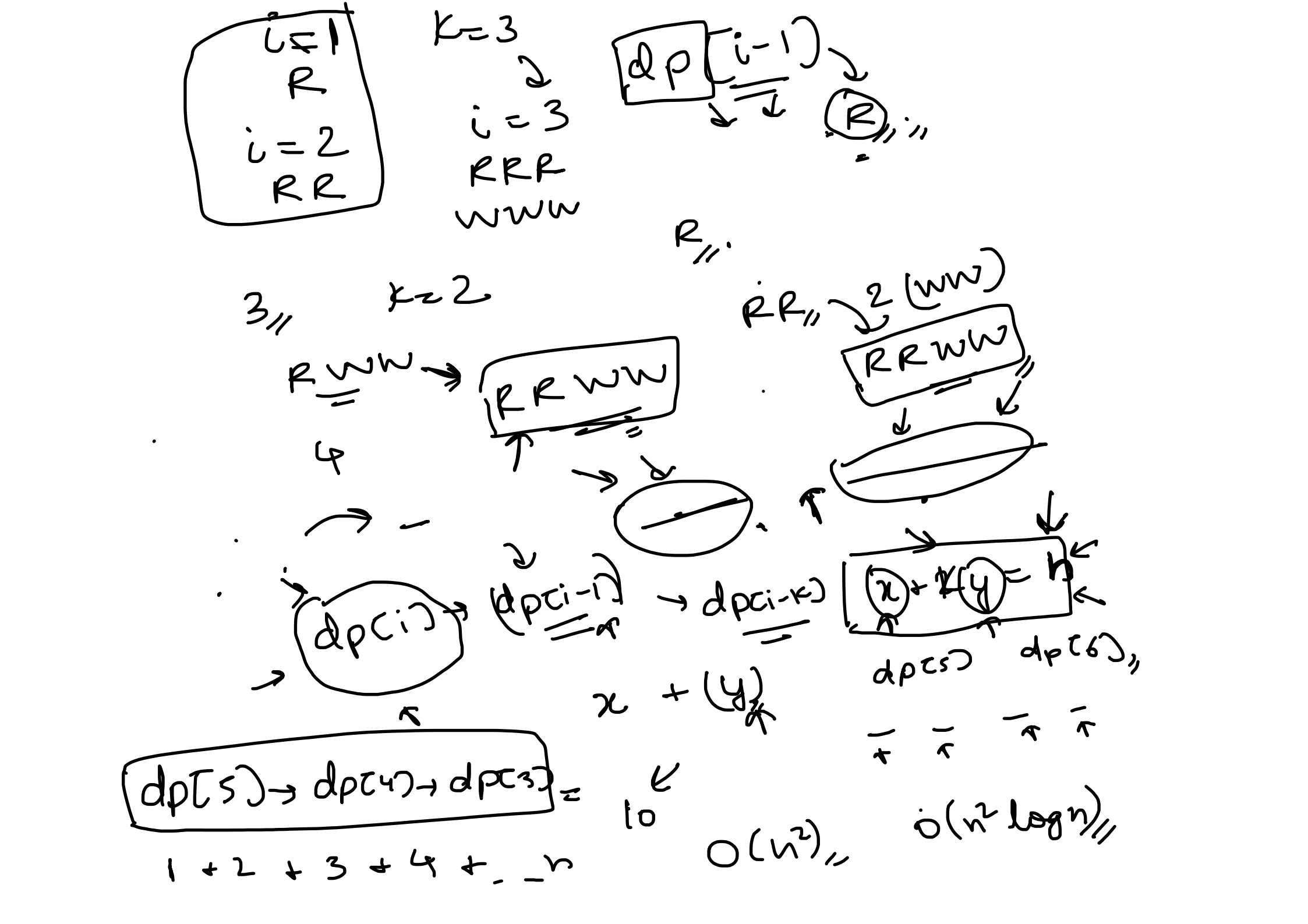


k = 2



dp[i] → dp[i+1]
→ dp[i+k]

dp[i] ← +1 dp[i-1]
dp[i] ← k dp[i-k]
k whites.


i=1
R
i=2
RR
k=3
i=3
RRR
WWW
dp(i-1)
R
R//
3//
k=2
RWW
RRWW
RR//
2 (WW)
RRWW
4
dp(i)
dp(i-1)
dp(i-k)
x + (y) = h
dp(5)
dp(6)//
x + (y)
dp[5] → dp[4] → dp[3] =
1 + 2 + 3 + 4 + _ _ n
10
O(n²)//
O(n² log n)//

dp[0] → 1 // 0 =

dp[i] → dp[i-1] //

i<K → 0

dp[1] → dp[0] //

k=3

dp[1] → dp[0] + 1 //

dp[2] → dp[1] + 1

2! — —

→ (2) //

RR //

R KW //

— — — — —

← n →

2^n → {3} //

K=3

2 → 1 //

RR R

R

0! = 1 //

0 → 1 //

{3} → 1 //

i-1 i-2 . . .

dp[i] → dp[i-1] //

dp[i] → dp[i-K] //

K white

2 K=2 . R .

0 1 2 3 n RR WW

③ RRR WWR RWW

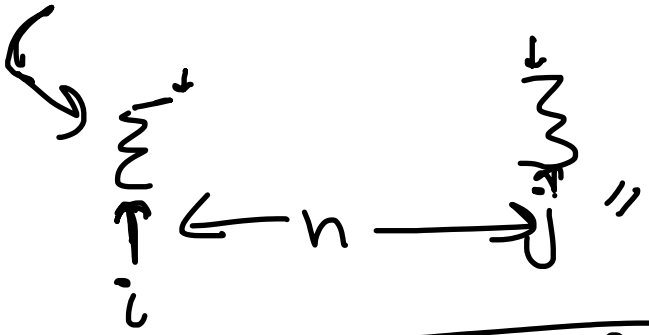
Problem 2 [CSES]

Removal Game:
https://cses.fi/problemset/task/1097

Main Idea

- The dp function can calculate the optimal values for states of only a single player.
- Therefore, we must simulate the optimality of the second player on our own.
- We can do this by considering both cases of the second person once the move of the first person is chosen and then take the maximum of those two.
- In this way, the dp is called always 2 steps further.

1st person



$i = 0$, $j = n-1$

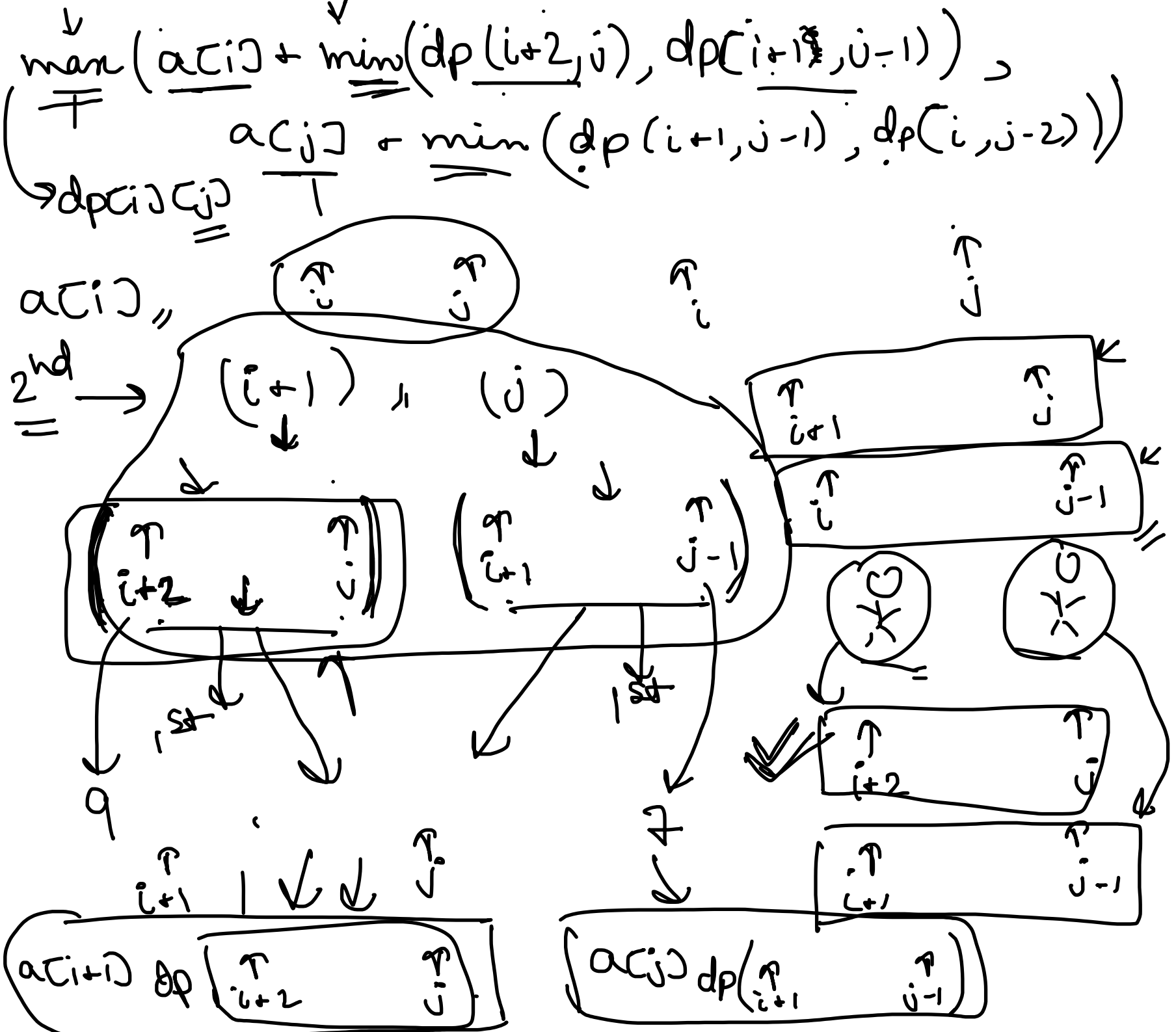
1st 2nd 1st

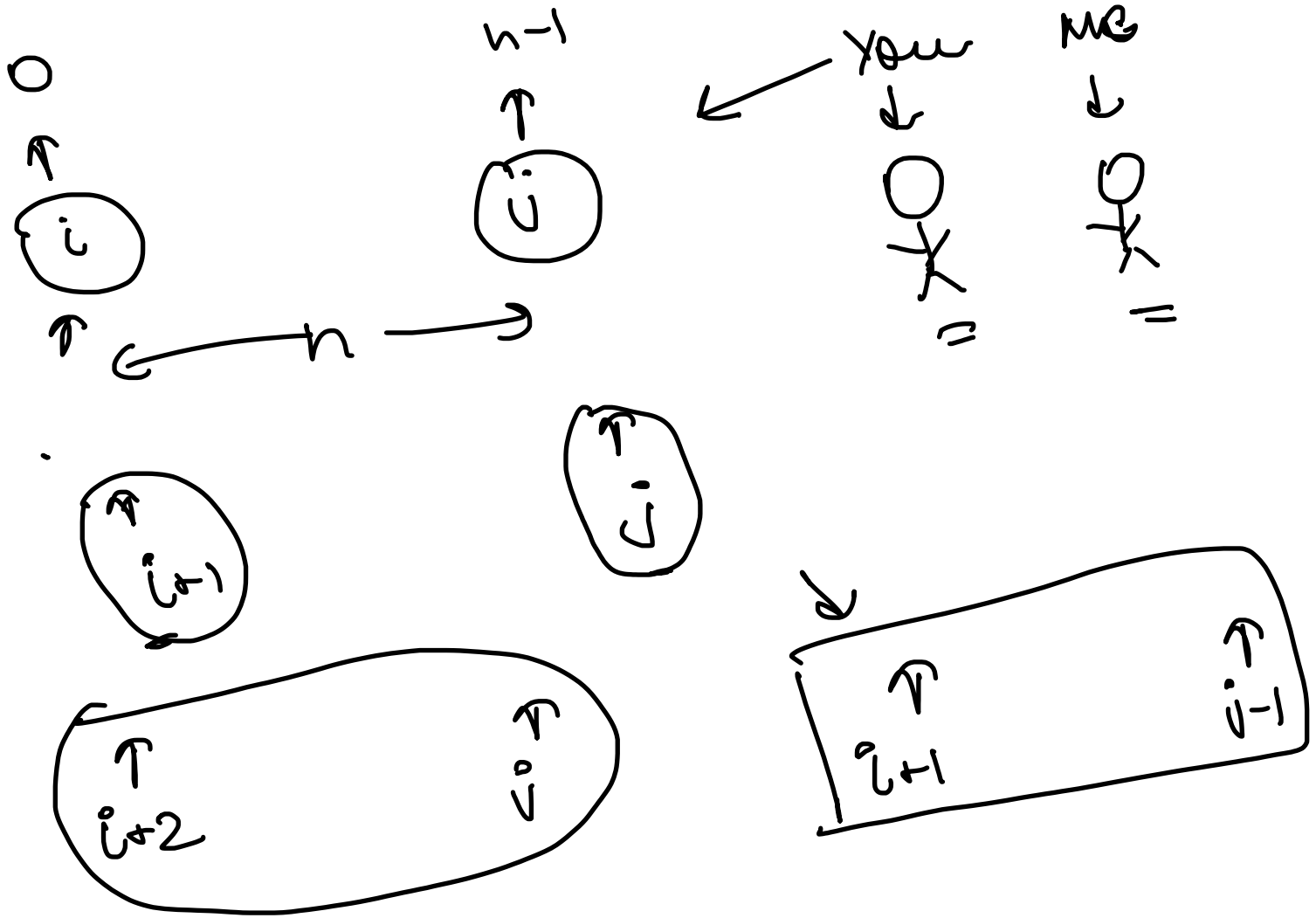
a[i] → a[i+1] or a[j]

a[j] → a[i] or a[j-1]

max(a[i] + dp(3rd turn) , a[j] + dp(3rd turn))

2nd


max(a[i] + min(dp(i+2, j), dp(i+1, j-1)),
a[j] + min(dp(i+1, j-1), dp(i, j-2)))
dp[i][j]
a[i],
2nd
(i+1), (j)
i+2
j
i+1
j-1
i
1st
1st
9
7
a[i+1] dp
a[j] dp



$S_1 + S_2 =$ constant

$=$ sum of array

Problem 3 [Atcoder DP Contest]

Knapsack 2:
https://atcoder.jp/contests/dp/tasks/dp_e

Main Idea

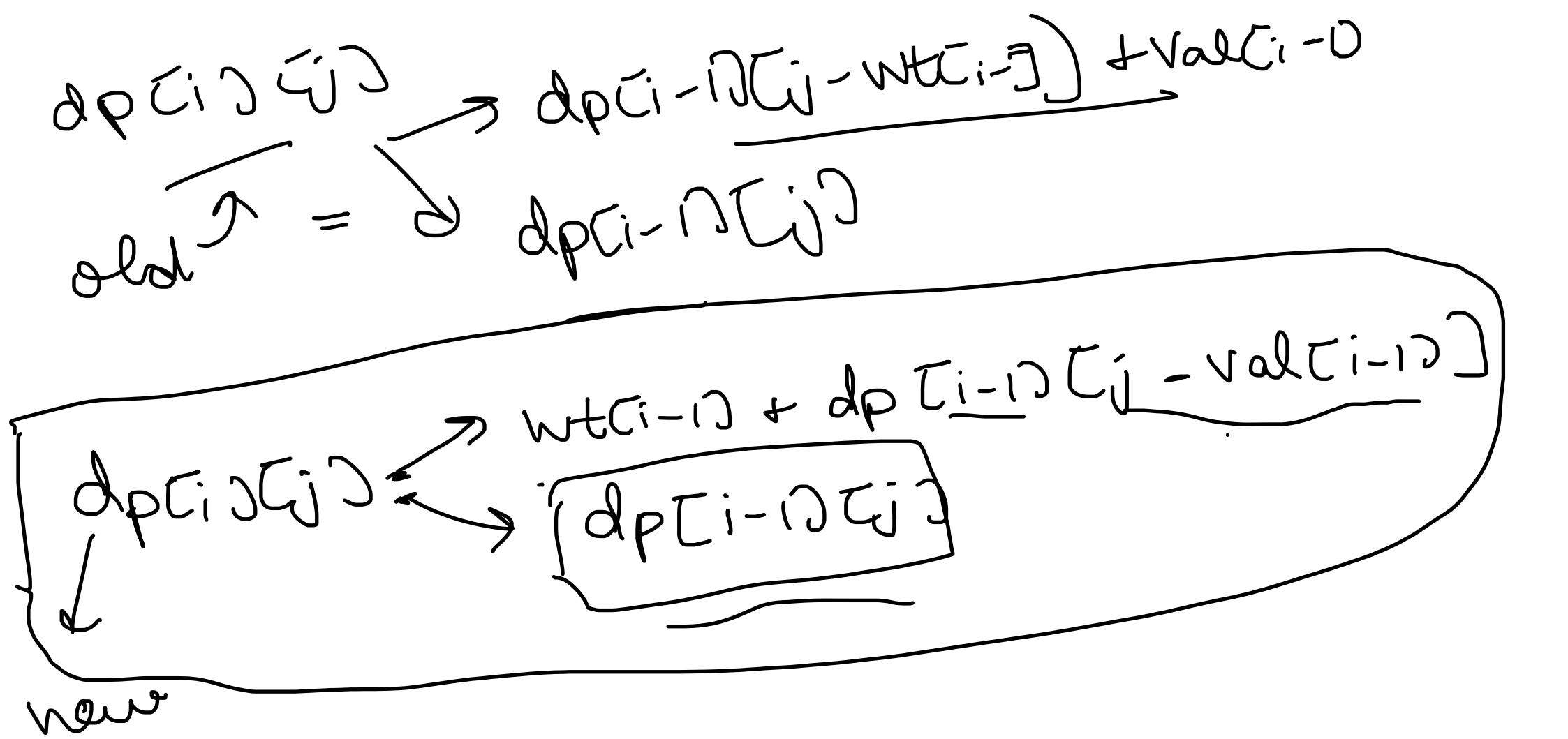
- Instead of taking the standard state space and transition (finding the maximum value for a given (**i**, **w**) pair, we will form a different pair of states.
- Find for every (**i**, **val**) pair, the minimum weight from which it could be obtained.
- This will work in the given constraints.
- Update the transitions according to this as well.

dp[i][j] →

maximum possible value
if we consider only first i items
and fill j units of weight.

dp[i][j] →

minimum weight to obtain j value
while considering first i items.


dp[i][j]
dp[i-1][j-wt[i-1]] + val[i-1]
old
=
dp[i-1][j]
dp[i][j]
wt[i-1] + dp[i-1][j-val[i-1]]
dp[i-1][j]
new